

Advanced Programming

CSE 201

Lecture 17: Inner Class & JUnit-1

Phone Batteries are Becoming Non-Removable



Non Removable Battery Vs. Removable Battery

- Nowadays most of the phones are coming with non removable batteries
- Earlier, when there were removable batteries, we could easily keep a spare battery and replace when the primary one drained!
 - *Yes, I know we can use a power bank today!*

Inner Class

```
public class SamsungGalaxy {  
    private FixedBattery myBattery;  
  
    public SamsungGalaxy() {  
        myBattery = new FixedBattery();  
    }  
  
    private class FixedBattery {  
        ....  
    }  
}
```

Inner Class

- Description
 - Class defined in scope of another class
- Property
 - Can directly access **all** variables & methods of enclosing class (including private fields & methods)
- Why inner class?
 - Logical grouping of functionality
 - Increases encapsulation
 - Consider two top-level classes, A and B, where B needs access to members of A that would otherwise be declared private. By hiding class B within class A, A's members can be declared private and B can access them. In addition, B itself can be hidden from the outside world
 - More readability and maintainable code

Inner Class Instance Inside a Method of Outer Class

```
public class SamsungGalaxy {  
  
    private FixedBattery myBattery;  
  
    public SamsungGalaxy() {  
        myBattery = new FixedBattery();  
    }  
  
    private class FixedBattery {  
        private boolean runDiagnosis() { ..... }  
        ....  
    }  
  
    public static void main(String[] args) {  
        SamsungGalaxy sg = new SamsungGalaxy();  
        SamsungGalaxy.FixedBattery sgb  
            = sg.new FixedBattery();  
        boolean test = sgb.runDiagnosis();  
    }  
}
```

- To instantiate the inner class in some other class, first we have to instantiate the outer class
- Thereafter, using the object of the outer class, we can instantiate the inner class
 - Note the usage of “new” keyword

Inner Class Instance Outside Outer Class

```
public class SamsungGalaxy {  
  
    private FixedBattery myBattery;  
  
    public SamsungGalaxy() {  
        myBattery = new FixedBattery();  
    }  
  
    private class FixedBattery {  
        private boolean runDiagnosis() { ..... }  
        .....  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        SamsungGalaxy sg = new SamsungGalaxy();  
        SamsungGalaxy.FixedBattery sgb  
            = sg.new FixedBattery();  
        boolean test = sgb.runDiagnosis();  
    }  
}
```

- Is this code correct?
 - NO, compilation error!
- Inner class FixedBattery is private and hence cannot be accessed in another class
 - Rules of “private” modifier
 - Making the inner class and its method as public will correct this code

```
class SamsungGalaxy {
    private int version=2;
    private FixedBattery myBattery;

    public SamsungGalaxy() {
        myBattery = new FixedBattery();
    }

    public class FixedBattery {
        private int version=3;
        public void print() {
            int version = 4;
            System.out.println(version);
            System.out.println(this.version);
            System.out.println(SamsungGalaxy.this.version);
        }
    }
}

class Main {
    public static void main(String[] args) {
        SamsungGalaxy sg = new SamsungGalaxy();
        SamsungGalaxy.FixedBattery sgb = sg.new FixedBattery();
        sgb.print();
    }
}
```

Output = 4, 3, 2

Bugs and Testing

- **Software reliability:** Probability that a software system will not cause failure under specified conditions.
- **Bugs** are inevitable in any complex software system.
 - Industry estimates: 10-50 bugs per 1000 lines of code.
 - A bug can be visible or can hide in your code until much later.
- **Testing:** A systematic attempt to reveal errors.
 - Failed test: an error was demonstrated.
 - Passed test: no error was found (for this particular situation)
 - It plays a critical role in helping a software company deliver quality products to its customers.



Manual Testing v/s Automated Testing

Manual Testing	Automated Testing
Executing a test cases manually without any tool support is known as manual testing	Taking tool support and executing the test cases by using an automation tool is known as automation testing
Time-consuming and tedious – Since test cases are executed by human resources, it is very slow and tedious	Fast – Automation runs test cases significantly faster than human resources
Huge investment in human resources – As test cases need to be executed manually, more testers are required in manual testing	Less investment in human resources – Test cases are executed using automation tools, so less number of testers are required in automation testing
Less reliable – Manual testing is less reliable, as it has to account for human errors	More reliable – Automation tests are precise and reliable.
Non-programmable – No programming can be done to write sophisticated tests to fetch hidden information	Programmable – Testers can program sophisticated tests to bring out hidden information

JUnit: Java Unit Testing Framework



- The Java library **JUnit** helps us to easily perform automated unit testing
- The basic idea:
 - For a given class `Foo`, create another class `FooTest` to test it, containing various "test case" methods to run.
 - Each method looks for particular results and passes / fails
- JUnit provides "**assert**" commands to help us write tests.
 - The idea: Put assertion calls in your test methods to check things you expect to be true. If they aren't, the test will fail

Sample JUnit Test

```
/* The class method to be tested */
public class Sum {
    private int var1, var2;
    public Sum(int v1, int v2) {var1=v1; var2=v2;}
    public int sum () {
        return var1 + var2;
    }
}
```

```
/* Junit test class */

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class MyTest {

    @Test
    public void testSum() {
        Sum mySum = new Sum(1, 1);
        int sum = mySum.sum();
        assertEquals(2, sum);
    }
}
```

```
/* Junit test runner class */

import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

public class TestRunner {
    public static void main(String[] args) {
        Result result= JUnitCore.runClasses(MyTest.class);
        for (Failure failure : result.getFailures()) {
            System.out.println(failure.toString());
        }
        System.out.println(result.wasSuccessful());
    }
}
```